Real-Time End-to-End Autonomous Driving in AlpaSim

Myra Cropper
Electrical and Computer Engineering
mcropper@andrew.cmu.edu

1 Problem Statement

Models such as NVIDIA's AlpamayoR1 (1) and Alpamayo1.5 (1) have achieved state of the art closed-loop driving performance by incorporating explicit reasoning into the driving pipeline through chain-of-thought (CoT) reasoning and diffusion-based trajectory planning. Unlike traditional end-to-end driving policies that directly map sensor observations to control actions, reasoning-based Vision-Language-Action (VLA) models generate intermediate semantic representations that allow the model to interpret complex traffic scenarios, justify its decisions, and improve generalization to long tail driving situations.

The reasoning capability addresses several limitations of conventional Vision-Language Models (VLMs). VLMs inherit many of the shortcomings of Large Language Models (LLMs), including weak spatial reasoning, poor detail recognition, and difficulty understanding logical negation (2; 3). Furthermore, their deep transformer architectures are prone to cascading reasoning errors, where mistakes introduced in early layers propagate throughout the network, potentially leading to unsafe or inexplicable driving decisions. Recent work has shown that CoT reasoning significantly improves an LLM's ability to perform complex multi-step reasoning tasks (4). In autonomous driving, approaches such as DriveVLM (5) leverage LLM based planners to generate structured reasoning traces alongside trajectory predictions, improving both transparency and auditability compared to purely black box policies. Alpamayo extends this reasoning-based paradigm to end-to-end autonomous driving, enabling stronger closed loop performance while providing interpretable decision making.

However, these improvements come at a substantial computational cost. The large transformer backbones, explicit reasoning process, and diffusion-based trajectory planners used by reasoning-based VLA models introduce significant inference latency, making it challenging to satisfy the real-time constraints required for autonomous driving deployment.

The NVIDIA AlpaSim Closed-Loop Challenge introduces an additional systems constraint beyond driving accuracy. Participating policies must produce a control action within a **100 ms inference budget**, corresponding to a **10 Hz control loop**. Policies that exceed this latency budget cannot operate in real time and receive reduced challenge scores due to throughput violations. Consequently, inference efficiency becomes as important as driving performance.

We will investigate:

> *How can reasoning-based Vision-Language-Action driving models be architecturally decomposed and computationally optimized to reduce inference cost while preserving closed-loop driving performance within a 100 ms control budget?*

We will first establish reproducible performance baselines for Alpamayo1.5, SIMFORGE-D1.5(6), and FlashDrive within the AlpaSim (7) framework. Building upon these baselines, we will investigate visual feature caching, temporal separation of perception and reasoning, visual token pruning, and lightweight trajectory generation (strength goal). The goal is to determine whether these architectural modifications can reduce inference latency and computational requirements while maintaining closed-loop driving performance.

.

2 Data

All experiments will be conducted using NVIDIA's AlpaSim(7) closed-loop simulator. AlpaSim provides a high-fidelity autonomous driving benchmark consisting of diverse urban driving scenarios, realistic traffic participants, dynamic environments, and standardized evaluation metrics. The simulator enables reproducible closed-loop evaluation by allowing driving policies to interact directly with the simulated environment.

3 Method

3.1 Baseline Evaluation

We will first establish reproducible baseline performance by evaluating NVIDIA's Alpamayo1.5 (1), SIMFORGE-D1.5(6), and FlashDrive within the AlpaSim (7) framework. The models provide representative approaches for evaluating the computational cost and closed-loop performance of reasoning-based and real-time end-to-end driving policies.

For each baseline policy, we will measure:

- Inference latency (ms)
- Component-level inference latency
- Simulator throughput (FPS)
- Route completion
- Collision rate
- Overall driving score

We will additionally determine whether each policy satisfies the 100 ms closed-loop constraint and measure the percentage of inference cycles that satisfy this requirement.

3.2 Visual Feature Cache Compression

Consecutive driving frames contain substantial temporal redundancy, requiring existing end-to-end driving models to repeatedly process similar visual information. We will investigate a visual feature cache that stores the output of the vision encoder and reuses visual representations across consecutive frames.

Rather than recomputing the complete visual representation at every control timestep, the proposed system will maintain a temporally persistent cache of visual features. New observations will update the cache while redundant features are reused. We will investigate different cache sizes and update strategies and evaluate their effects on vision encoder latency, GPU memory utilization, inference throughput, and closed-loop driving performance.

3.3 Decoupled Fast Perception and Slow Reasoning

Building upon visual feature caching, we will investigate separating high-frequency perception and trajectory generation from lower-frequency semantic reasoning. The motivation is that the computationally expensive reasoning model does not necessarily need to execute at every 100 ms control interval.

The proposed architecture will maintain two computational pathways. A fast perception pathway will operate at the control frequency and maintain an updated visual representation. A slower reasoning pathway will periodically execute the larger Vision-Language-Action model to generate a high-level representation of the desired driving behavior.

The output of the long-term reasoning module will be converted into a compact desire vector representing the desired driving behavior. This desire vector will be provided as an input to a lightweight path-planning model together with the current visual representation and ego-vehicle state. The lightweight planner will generate high-frequency trajectory updates without requiring the full reasoning model to execute at every control timestep.

We will evaluate different reasoning frequencies and measure the resulting tradeoff between computational cost and closed-loop driving performance.

3.4 Visual Token Pruning

We will investigate reducing the computational cost of the vision-language interface through visual token pruning. The vision encoder will produce a set of visual tokens, from which a subset will be selected according to token relevance.

Experiments will evaluate different token-retention ratios and measure their effects on inference latency, GPU memory utilization, and closed-loop driving performance. These experiments will determine the extent to which redundant visual information can be removed without substantially degrading driving behavior.

3.5 (Stretch goal) Lightweight Trajectory Generation

As a later architectural modification, we will investigate replacing the computationally expensive diffusion-based trajectory planner with a lightweight single-pass trajectory prediction head.

The proposed planner will receive the desire vector produced by the long-term reasoning module, together with the current visual representation and ego-vehicle state, and directly predict the future vehicle trajectory. The trajectory can be represented as a sequence of future waypoints:

$$\tau = f_\theta \left(z_{\text{desire}}, z_{\text{visual}}, s_{\text{ego}} \right), \tag{1}$$

where z_{desire} represents the high-level driving intent, z_{visual} represents the current visual state, s_{ego} represents the ego-vehicle state, and τ represents the predicted future trajectory.

We will evaluate whether a lightweight single-pass trajectory predictor can replace the iterative diffusion-based planner while maintaining closed-loop driving performance and substantially reducing trajectory generation latency.

4 Deliverables

As a result of this work, we will deliver:

- A reproducible AlpaSim benchmarking framework for Alpamayo1.5, SIMFORGE-D1.5, and FlashDrive.
- A detailed comparison of baseline inference latency, computational requirements, and closed-loop driving performance.
- A visual feature caching implementation for reducing redundant vision encoder computation across consecutive frames.
- A decoupled fast-perception and slow-reasoning architecture that uses a compact desire vector to communicate high-level driving intent to a lightweight trajectory planner.
- An evaluation of visual token pruning for reducing the computational cost of visual representations.
- An experimental evaluation of a lightweight single-pass trajectory prediction head as an alternative to diffusion-based trajectory generation.
- An empirical analysis of the tradeoffs between inference latency, GPU memory utilization, and closed-loop driving performance.
- A final technical report summarizing the methodology, experimental results, and recommendations for future work.

References

[1] NVIDIA, Y. Wang, W. Luo, J. Bai, Y. Cao, T. Che, K. Chen, Y. Chen, J. Diamond, Y. Ding, W. Ding, L. Feng, G. Heinrich, J. Huang, P. Karkus, B. Li, P. Li, T.-Y. Lin, D. Liu, M.-Y. Liu,

L. Liu, Z. Liu, J. Lu, Y. Mao, P. Molchanov, L. Pavao, Z. Peng, M. Ranzinger, E. Schmerling, S. Shen, Y. Shi, S. Tariq, R. Tian, T. Wekel, X. Weng, T. Xiao, E. Yang, X. Yang, Y. You, X. Zeng, W. Zhang, B. Ivanovic, and M. Pavone, "Alpamayo-R1: Bridging reasoning and action prediction for generalizable autonomous driving in the long tail," *arXiv preprint arXiv:2511.00088*, 2025.

[2] S. Chen, T. Zhu, R. Zhou, J. Zhang, S. Gao1, J. C. Niebles, M. Geva, J. He, J. Wu, and M. Li, "Why is spatial reasoning hard for vlms? an attention mechanism perspective on focus areas," *arXiv preprint arXiv:2503.01773*, 2025.

[3] K. Alhamoud, S. Alshammari, Y. Tian, G. Li, P. H. Torr, Y. Kim, and M. Ghassemi, "Vision-language models do not understand negation," *Computer Vision and Pattern Recognition Conference*, 2025.

[4] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H.Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models,"

[5] X. Tian, J. Gu, B. Li, Y. Liu, Y. Wang, Z. Zhao, K. Zhan, P. Jia, X. Lang, and H. Zhao, "Drivevlm: The convergence of autonomous driving and large vision-language models," *arXiv preprint arXiv:2402.12289*, 2024.

[6] SIMFORGE-AI, "SIMFORGE-D1.5." `https://huggingface.co/SIMFORGE-AI/SIMFORGE-D1.5`, 2025. Accessed: Sep. 21, 2026.

[7] NVIDIA, Y. Cao, R. de Lutio, S. Fidler, G. G. Cobo, Z. Gojcic, M. Igl, B. Ivanovic, P. Karkus, J. M. Esturo, M. Pavone, A. Smith, E. Tanimura, M. Tyszkiewicz, M. Watson, Q. Wu, and L. Zhang, "Alpasim: A modular, lightweight, and data-driven research simulator for autonomous driving," October 2025.